

Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки і комп'ютерних технологій

Звіт
про виконання лабораторної роботи №2
з курсу «Хмарні технології та великі дані»
на тему:
«Debounce»

Виконав:
студент групи ФсІ-42
Трофимов Володимир
Перевірив:
ас. Керод Т. І.

Зміст

Тема.....	3
Мета	3
Вхідні дані	3
Завдання.....	3
Критерії виконання роботи.....	4
Теоретичні відомості	6
Структура програмної реалізації	6
Тестування програмної реалізації.....	7
Висновок	10
Код програми	11

Тема

Зменшення кількості викликів функції під час частих подій за допомогою патерну Debounce.

Мета

Дослідження принципу роботи патерну Debounce та його реалізація мовою TypeScript із використанням середовища виконання Node.js. Необхідно реалізувати механізм відкладеного виконання функції, який обмежує кількість фактичних викликів під час послідовності швидких подій, забезпечити підтримку режимів leading і trailing, а також перевірити коректність роботи реалізації шляхом ручної демонстрації на прикладі перекладу введеного користувачем тексту.

Вхідні дані

Параметр	Значення	Призначення
delayMs	500 мс	час очікування після останньої події перед виконанням функції
leading	False	визначає, чи повинна функція виконуватися одразу при першому виклику серії
trailing	True	визначає, чи повинна функція виконуватися після завершення серії швидких викликів
text	введений користувачем текст	дані, які передаються до функції перекладу
sourceLanguage	EN	мова вихідного тексту
targetLanguage	UK	мова перекладу

Завдання

Розробити програмну реалізацію патерну Debounce мовою TypeScript для обмеження кількості фактичних викликів функції під час виникнення великої кількості послідовних подій.

Реалізація повинна забезпечувати:

- створення Debounce-механізму з параметром delayMs;
- перезапуск таймера після кожного нового виклику;
- збереження останніх переданих аргументів для trailing-виконання;
- підтримку режиму leading, у якому функція виконується одразу при першому виклику серії;

- підтримку режиму trailing, у якому функція виконується після завершення серії викликів;
- підтримку одночасного використання leading і trailing;
- коректну роботу при вимкнених leading та trailing;
- реалізацію методу dispose() для скасування відкладеного виконання;
- коректне очищення таймерів та внутрішнього стану;
- підрахунок кількості отриманих викликів та фактичних виконань;
- розмежування подій CALL, EXECUTION та HTTP-запитів;
- інтеграцію Debounce із текстовим полем вебінтерфейсу;
- виконання перекладу лише під час фактичного виконання Debounce-функції;
- реалізацію Node.js + Express backend;
- взаємодію backend із зовнішнім сервісом перекладу DeepL API;
- передачу API-ключа через змінну середовища без його розміщення у frontend-кодi;
- демонстрацію роботи всіх режимів Debounce за допомогою вебінтерфейсу.

Критерії виконання роботи

Лабораторна робота вважається виконаною за умови реалізації та ручної перевірки основної функціональності патерну Debounce.

1. Реалізація механізму Debounce

Програмна реалізація повинна містити окремий механізм Debounce, який приймає функцію та параметри налаштування і повертає функцію-обгортку. При кожному виклику обгортки таймер повинен скидатися та запускатися заново.

2. Робота параметра delayMs

Параметр delayMs повинен визначати проміжок часу, протягом якого після останнього виклику функція не виконується у режимі trailing. Наприклад, при delayMs = 1000 мс функція повинна виконатися через одну секунду після останньої події.

3. Робота режиму leading

При leading = true перший виклик серії повинен призводити до негайного виконання функції. Наступні виклики протягом встановленого інтервалу повинні блокуватися. Після завершення періоду Debounce новий виклик повинен мати можливість розпочати нову серію.

4. Робота режиму trailing

При trailing = true функція повинна виконуватися після завершення серії швидких викликів. При кожній новій події таймер повинен перезапуститися, тому виконується саме останній виклик із відповідними аргументами.

5. Комбінація *leading* та *trailing*

При одночасному встановленні *leading* = true та *trailing* = true перший виклик повинен виконуватися негайно, а після серії наступних викликів — останній виклик повинен виконуватися після завершення періоду очікування. При цьому для одного ізольованого виклику не повинно відбуватися дублювання: *leading*-виконання не повинно супроводжуватися зайвим *trailing*-виконанням.

6. Робота при *leading* = false та *trailing* = false

Якщо обидва параметри вимкнені: *leading* = false та *trailing* = false функція не повинна виконуватися. При цьому самі події CALL можуть реєструватися у журналі для демонстрації роботи Debounce.

7. Метод *dispose()*

Метод *dispose()* повинен:

- скасовувати активний таймер;
- видаляти збережені аргументи;
- скидати внутрішній стан Debounce;
- запобігати виконанню запланованого *trailing*-виклику;
- коректно працювати при повторному виклику.

8. Інтеграція з HTTP-запитами

HTTP-запит до *backend* не повинен виконуватися при кожній події введення.

9. Відображення результатів

Вебінтерфейс повинен відображати:

- введений текст;
- результат перекладу;
- значення *delayMs*;
- параметри *leading* та *trailing*;
- журнал подій;
- кількість CALL;
- кількість фактичних EXECUTION;
- кількість HTTP-запитів;
- кількість пригнічених викликів.

10. Демонстрація роботи

Коректність реалізації повинна бути продемонстрована вручну за допомогою вебінтерфейсу.

Необхідно показати:

- `leading = false, trailing = true;`
- `leading = true, trailing = false;`
- `leading = true, trailing = true;`
- `leading = false, trailing = false;`

скасування відкладеного виконання за допомогою `dispose()`.

Особливу увагу необхідно звернути на співвідношення:

$$\text{CALLS} > \text{EXECUTIONS} \geq \text{HTTP REQUESTS}$$

у випадку швидкого введення тексту.

Теоретичні відомості

Debounce — це програмний патерн керування частими подіями, який дозволяє обмежити кількість фактичних виконань функції при великій кількості послідовних викликів.

Основний принцип Debounce полягає в тому, що після кожного виклику запускається таймер. Якщо до завершення встановленого інтервалу надходить новий виклик, попередній таймер скасовується і запускається заново. Таким чином, функція може бути виконана лише після припинення потоку подій протягом заданого часу.

Цей механізм особливо корисний для обробки подій введення тексту, пошуку, зміни параметрів інтерфейсу та інших операцій, які можуть генерувати велику кількість подій за короткий проміжок часу.

Параметр `delayMs` визначає тривалість періоду очікування. Параметр `leading` визначає можливість негайного виконання функції на початку серії викликів, а `trailing` — виконання останнього виклику після завершення серії.

У лабораторній роботі Debounce використовується для керування запитами перекладу. Кожна зміна текстового поля є окремим CALL, однак HTTP-запит до сервера виконується лише після фактичного EXECUTION Debounce-функції. Це дозволяє уникнути відправлення окремого запиту до сервісу перекладу для кожного введенного символу.

Структура програмної реалізації

Програмна реалізація лабораторної роботи виконана мовою TypeScript із використанням середовища виконання Node.js та фреймворку Express.

Програма складається з трьох основних частин: реалізації механізму Debounce, клієнтського вебінтерфейсу та серверної частини для виконання перекладу.

Клієнтська частина реагує на події введення тексту. Кожна така подія передається до Debounce як окремий виклик. Після завершення періоду очікування відповідно до встановлених параметрів виконується функція перекладу.

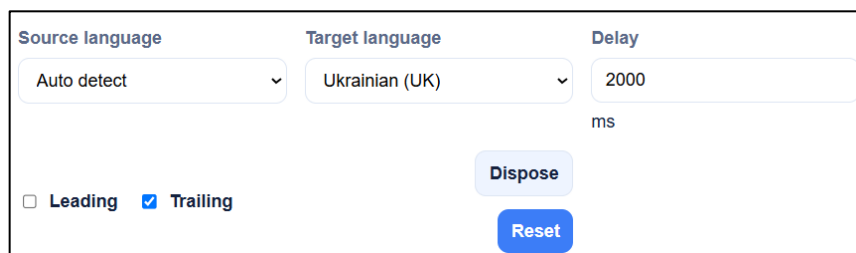
Серверна частина реалізована за допомогою Node.js та Express. Вона приймає HTTP-запити від клієнта через endpoint `/api/translate`, виконує запит до DeepL API та повертає отриманий переклад клієнтській частині.

API-ключ DeepL зберігається у змінній середовища `.env`, тому він не передається у frontend та не розкривається користувачу.

Тестування програмної реалізації

Тестування реалізації виконується вручну за допомогою вебінтерфейсу, без використання автоматизованих тестових фреймворків. Першим етапом є запуск Node.js-сервера та відкриття вебінтерфейсу лабораторної роботи. Далі встановлюються параметри:

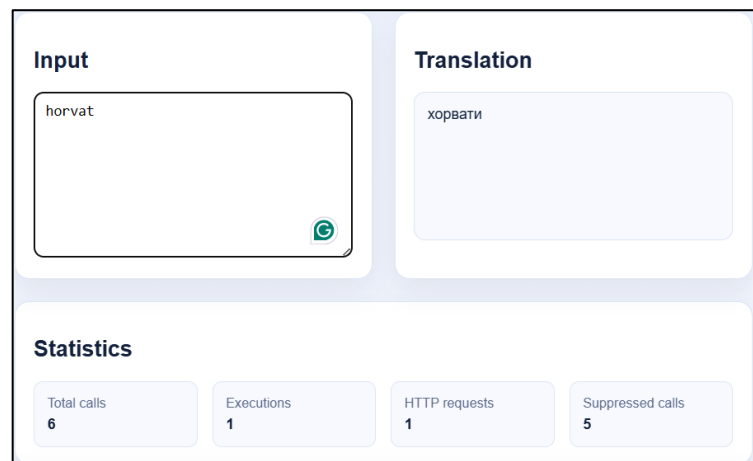
- `delayMs = 1000`
- `leading = false`
- `trailing = true`



The image shows a configuration interface for a debounce function. It includes three input fields: 'Source language' set to 'Auto detect', 'Target language' set to 'Ukrainian (UK)', and 'Delay' set to '2000 ms'. Below these are checkboxes for 'Leading' (unchecked) and 'Trailing' (checked). There are also 'Dispose' and 'Reset' buttons.

Рис.1 – Встановлення параметрів

Після цього користувач швидко вводить текст. У журналі фіксується декілька подій CALL, однак після припинення введення виконується лише один TRAILING EXECUTION, після якого формується один HTTP-запит до backend.



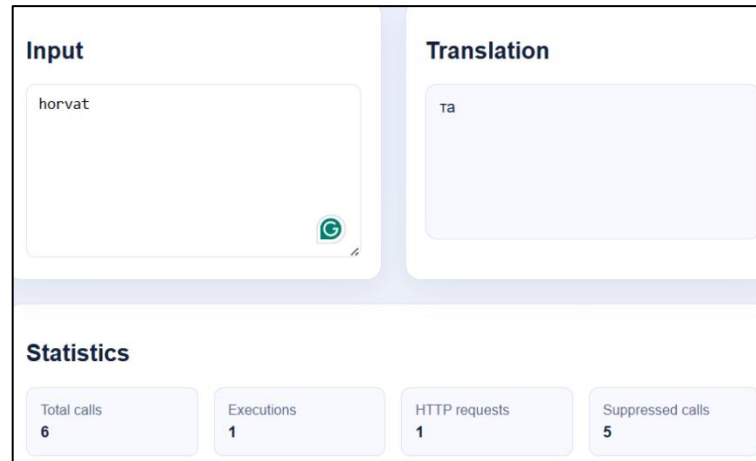
The image shows the application interface. It has an 'Input' section with a text box containing 'horvat' and a green circular icon. To the right is a 'Translation' section with a text box containing 'хорвати'. At the bottom is a 'Statistics' section with four boxes: 'Total calls' (6), 'Executions' (1), 'HTTP requests' (1), and 'Suppressed calls' (5).

*Рис.2 – Робота Debounce при параметрах `delayMs = 1000`, `leading = false`
`trailing = true`*

Наступним експериментом встановлюється:

- leading = true
- trailing = false

У цьому режимі перший виклик виконується негайно, а наступні виклики протягом періоду Debounce пригнічуються.

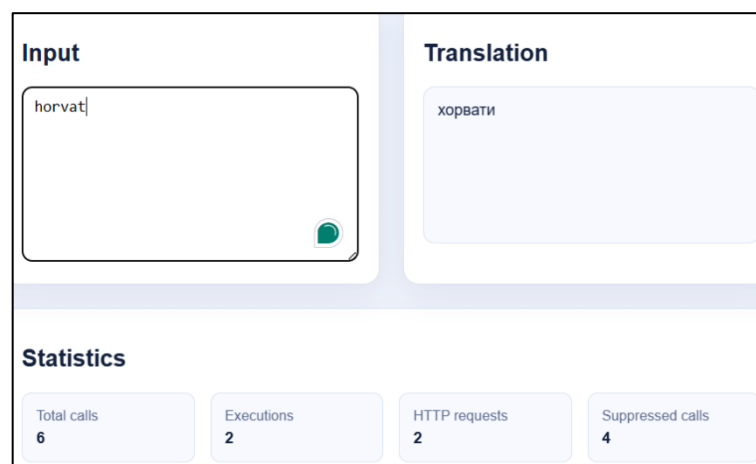


*Рис.3 – Робота Debounce при параметрах $delayMs = 1000$, $leading = true$
 $trailing = false$*

Далі перевіряється комбінація:

- leading = true
- trailing = true

Перший виклик виконується негайно, а після серії швидких викликів виконується останній виклик. Таким чином, демонструється одночасна робота обох режимів..



*Рис.4 – Робота Debounce при параметрах $delayMs = 1000$, $leading = true$
 $trailing = true$*

Також перевіряється конфігурація:

- leading = false
- trailing = false

У цьому випадку фактичне виконання функції не відбувається.

The screenshot displays a web application interface with three main sections: 'Input', 'Translation', and 'Statistics'. The 'Input' section contains a text box with the text 'horvat' and a green circular icon with a white 'G' at the bottom right. The 'Translation' section shows a light blue box with the text 'Waiting for input...'. The 'Statistics' section at the bottom features four light blue boxes with the following data: 'Total calls' with a value of 6, 'Executions' with a value of 0, 'HTTP requests' with a value of 0, and 'Suppressed calls' with a value of 6.

Input		Translation	
horvat		Waiting for input...	

Statistics			
Total calls	Executions	HTTP requests	Suppressed calls
6	0	0	6

*Рис.5 – Робота Debounce при параметрах $delayMs = 1000$, $leading = false$
 $trailing = false$*

Окремо перевіряється метод `dispose()`. Після створення відкладеного `trailing`-виконання викликається `dispose()`, після чого запланована операція не повинна виконуватися.

За результатами ручної перевірки аналізується журнал подій та порівнюється кількість CALL, EXECUTION і HTTP-запитів.

Висновок

У ході виконання лабораторної роботи було досліджено принцип роботи патерну Debounce та реалізовано його мовою TypeScript із використанням середовища виконання Node.js і фреймворку Express.

Було реалізовано механізм відкладеного виконання функції з параметром `delayMs`, а також підтримкою режимів `leading` і `trailing`. Реалізація забезпечує перезапуск таймера при кожному новому виклику, збереження останніх аргументів, пригнічення надлишкових викликів та скасування відкладених операцій за допомогою методу `dispose()`.

Для практичної демонстрації Debounce було створено вебінтерфейс, у якому кожна зміна текстового поля є окремим викликом Debounce. Фактичний запит на переклад формується лише під час виконання Debounce-функції. Для виконання перекладу використовується серверна частина на Node.js та Express, яка взаємодіє із зовнішнім сервісом DeepL API.

У результаті ручних експериментів було перевірено роботу окремих режимів `leading`, `trailing`, їх комбінації, режиму з обома параметрами, вимкненими, а також механізму `dispose()`. Було підтверджено, що Debounce дозволяє суттєво зменшити кількість фактичних виконань функції та HTTP-запитів під час швидкого потоку подій, зберігаючи при цьому можливість виконати операцію з актуальним останнім значенням.

Код програми

Код до даної лабораторної роботи розміщений у репозиторії https://github.com/MrCatman137/Cloud_Technologies у директорії Lab_2 та має повну інструкцію запуску даної програми.